

Design and Evaluation of a Basic Blockchain Implementation in Go

Max Eidet

Universitat Politècnica de Catalunya – Facultat
d’Informàtica de Barcelona
Barcelona, Spain
max.eidet@estudiantat.upc.edu

Olle Pettersson

Universitat Politècnica de Catalunya – Facultat
d’Informàtica de Barcelona
Barcelona, Spain
olle.pettersson@estudiantat.upc.edu

Abstract

This paper presents the design, implementation, and evaluation of a full-stack blockchain system built from scratch in Go with a React-based frontend explorer. The system implements SHA-256-based block linking, configurable Proof-of-Work consensus, ECDSA-signed transactions with nonce-based replay protection, a transaction mempool, and a decentralized peer-to-peer networking layer using UDP broadcast discovery and TCP communication. We evaluate the system through four experiments: mining performance under varying PoW difficulty, chain integrity under tampering, transaction validation under adversarial conditions, and multi-node synchronization behavior. Results show exponential mining-time growth with difficulty, successful detection of all chain tampering attempts, rejection of five distinct attack vectors (replay, signature forgery, public-key spoofing, overdraft, and future-timestamp manipulation), and correct chain convergence across networked nodes. The system demonstrates that a modular, relatively compact implementation can faithfully reproduce the core security and consensus properties of production blockchains while remaining accessible for experimentation and education.

1 Introduction

Blockchain technology has emerged as a fundamental component of decentralized systems, enabling trustless data exchange without reliance on centralized authorities. However, while blockchain systems are widely studied, their core mechanisms are often presented through simplified theoretical models that abstract away critical implementation details. As a result, there is a limited understanding of how these systems behave in practice, particularly with respect to networking, synchronization, and system-level trade-offs.

In particular, the interaction between consensus mechanisms, transaction propagation, and peer-to-peer communication remains difficult to study without a concrete implementation. Many educational models omit networking entirely, while production systems introduce significant complexity that obscures fundamental design principles. This creates a gap between conceptual understanding and practical system behavior.

In this work, we bridge this gap by designing and implementing a full-stack blockchain application that captures the essential components of a decentralized system while remaining accessible for experimentation. The system consists of a peer-to-peer blockchain node implemented in Go, combined with a web-based frontend built in React for interaction and visualization.

A central aspect of our design is the networking layer. Nodes operate in a fully decentralized environment, discovering peers via UDP-based broadcast and establishing TCP connections for reliable

communication. This approach enables nodes to propagate transactions, synchronize newly mined blocks, and maintain a shared view of the blockchain without centralized coordination. By explicitly separating lightweight discovery from reliable data transfer, the system exposes key trade-offs between efficiency and reliability in distributed systems.

To support interaction and observability, we integrate a frontend interface that allows users to visualize the blockchain, monitor node states, initiate mining, and submit transactions. This enables an end-to-end workflow, from user-generated transactions to block creation and network-wide propagation.

At a high level, we make the following contributions:

- Design and implementation of a modular blockchain system in Go, including block management, transaction handling, Proof-of-Work, and validation mechanisms.
- A decentralized peer-to-peer networking layer based on UDP discovery and TCP communication, enabling dynamic node discovery and synchronization.
- A web-based frontend for interacting with the system and visualizing blockchain state.
- An experimental evaluation of system behavior, including, chain integrity, transaction validation under adversarial conditions, and multi-node synchronization.

This paper addresses the following research questions:

- What architectural trade-offs are required to balance technical realism with educational clarity when implementing an interactive blockchain prototype?
- To what extent do cryptographic mechanisms, including ECDSA signatures, nonce ordering, and timestamp validation, protect against transaction, level attacks such as replay and forgery?
- What limitations arise from the use of a simplified peer-to-peer networking model?

2 Background

Traditional digital systems rely heavily on a trusted central authority, such as a bank or a single server, which introduces points of failure, censorship risks, and concentrated control. To eliminate this dependence, a blockchain serves as a distributed ledger that maintains a continuously growing list of records, called blocks, without relying on any central power source. Introduced by Nakamoto [5] as the underlying architecture for a peer-to-peer electronic cash system, this decentralized design enables a completely trustless environment. Instead of relying on a central intermediary, each participant in the network maintains a local copy of the ledger and verifies updates independently. Correctness and data integrity are

ultimately ensured through a combination of cryptographic mechanisms and consensus protocols, allowing multiple nodes to agree on a shared state while tolerating faulty or malicious participants.

2.1 Blockchain Structure

The core of a blockchain is the structure of its blocks and the way they are linked together [6]. Each block contains a set of transactions, a timestamp, and a cryptographic hash of the previous block. By including the previous hash, blocks are chained together in a sequential manner, forming a tamper-evident data structure. Any modification to a block would change its hash, thereby invalidating all subsequent blocks in the chain. This property ensures immutability, as altering historical data would require recomputing the hashes of all following blocks, which becomes computationally infeasible in practice [5].

2.2 Cryptographic Hashing

Cryptographic hash functions map arbitrary-length input to a fixed-length output with three key properties: determinism (the same input always produces the same hash), avalanche effect (a small change in input produces a drastically different output), and pre-image resistance (it is computationally infeasible to reverse the hash). SHA-256, a member of the SHA-2 family standardized by NIST, produces a 256-bit (32-byte) digest [7]. It is the primary algorithm used in Bitcoin and remains collision-resistant under current computational capabilities [5].

2.3 Proof-of-Work

The concept of using a computational hurdle as an artificial cost to deter resource abuse was originally proposed by Dwork and Naor [4]. In blockchain systems, this principle is adapted into Proof-of-Work (PoW), a consensus mechanism that requires miners to expend computational effort to produce a valid block [5]. Nodes connected to the network compete simultaneously to solve a cryptographic puzzle, which involves finding a block hash that satisfies a predetermined difficulty target. The first miner to discover a valid hash wins the right to append the next block to the blockchain and broadcast it to the network.

This process demands immense computational power, as miners must execute a brute-force search by iteratively changing a variable called a nonce until the resulting hash meets the strict structural criteria. Because cryptographic hashing functions are entirely deterministic yet unpredictable, there are no mathematical shortcuts. Therefore, the probability of finding a solution is directly proportional to the amount of computational work a node can perform. Consequently, this computational hurdle serves as a rate-limiting defense mechanism, making the modification of historical data economically and computationally prohibitive for malicious actors.

2.4 Transactions

In a blockchain architecture, a transaction represents a cryptographically secured transfer of value between two digital accounts. Rather than relying on physical identities, users interact with the network through cryptographic wallets, which consist of a public-private

key pair. The wallet address is derived directly from the user's public key, serving as their public identity on the network, while the private key is kept confidential and used to authorize actions.

To ensure that funds can only be moved by their rightful owner, transfer transactions are digitally signed using public-key cryptography. When a user initiates a transfer, their wallet uses the private key to generate a unique digital signature over the transaction details, which include the recipient's address, the amount, and a sequence number or nonce.

However, cryptographic signatures alone cannot prevent a sender from signing multiple conflicting transactions using the same funds. To thwart such double-spending attacks without a trusted central intermediary, all transactions must be publicly announced, requiring a mechanism for network participants to agree on a single, chronological history. This is achieved by organizing verified transactions into a linear, shared chain of blocks [5]. Consequently, a transaction is universally validated only when independent nodes collectively verify its chronological position against their local copy of the ledger, establishing a decentralized consensus that rejects any conflicting historical records.

2.5 Mempool

Once transactions are broadcast to the network, they require temporary storage before being mined [1]. The mempool (memory pool) acts as this staging area for unconfirmed transactions. In blockchain networks, when a node receives a new transaction, it validates the transaction against network consensus rules before adding it to its local pool. Miners then select transactions from this pool to bundle into new blocks. This decoupling between transaction submission and block creation is a fundamental architectural pattern in production blockchain systems.

2.6 Peer-to-Peer Networking

In a blockchain network, nodes communicate peer-to-peer (P2P) without relying on a central coordinator. To maintain the distributed ledger, nodes autonomously discover active peers and establish communication channels to propagate unconfirmed transactions and newly mined blocks. However, propagation is trustless, nodes independently validate all received transactions and blocks against network consensus rules before accepting or relaying them.

Due to network latency, nodes may receive competing blocks simultaneously, leading to temporary chain branches. To resolve this, networks employ a fork-resolution mechanism, typically the "longest chain rule," where nodes always consider the valid chain with the greatest accumulated proof-of-work to be the correct one. Additionally, the gossip-based protocol is inherently fault-tolerant. Broadcasts do not need to reach all nodes instantly, and nodes that miss broadcasts can request missing blocks from peers once they reconnect or detect a gap in their chain height [5].

2.7 Consensus

In a decentralized blockchain network, consensus protocols ensure that all independent nodes synchronize and agree on a single, authoritative version of the ledger without relying on a central coordinator. A foundational mechanism for achieving this in Proof-of-Work networks is the longest-chain rule [5].

Because multiple miners work on solving cryptographic puzzles simultaneously, the network will occasionally experience a temporary split, known as a fork, where two different valid blocks are discovered at roughly the same time. To resolve these conflicts and maintain a unified history, nodes inherently trust the chain that represents the greatest amount of accumulated proof or computational work. When a node detects that a peer possesses a longer valid chain, it will fetch the missing blocks and perform rigorous validation. This validation includes checking the cryptographic hashes, block sequencing, and individual transaction states. If the new chain is fully verified, the node adopts it, discarding its own shorter branch. This automatic realignment ensures that temporary network partitions are short-lived, forcing all honest nodes to quickly converge back onto a single, authoritative timeline.

2.8 Token Distribution Mechanisms

In blockchain networks, initial token distribution determines how cryptocurrency is introduced into circulation. While production networks often rely on continuous, algorithmic emission via mining rewards, alternative mechanisms are frequently utilized for bootstrapping networks or in test environments.

2.8.1 Pre-mining. Pre-mining refers to the process where a specific amount of a cryptocurrency's total supply is minted or allocated before the public launch of the network [3]. This approach is commonly used to provide immediate ecosystem liquidity or fund initial core project entities.

2.8.2 Crypto Faucets. A crypto faucet is an application or mechanism that distributes small amounts of cryptocurrency to users, typically without requiring financial compensation [2]. In development networks and testnets, faucets serve as critical infrastructure, enabling developers and testers to acquire tokens required for system interaction without economic cost.

3 System Design

The system follows a modular, multi-layered architecture organized into distinct components, each responsible for a well-defined aspect of the blockchain's functionality. The comprehensive full-stack architecture, detailing the interactions between the user-facing frontend, the local node's internal layers (API, Blockchain Core, and P2P Network Layer), and the broader decentralized peer-to-peer network, is visualized in Figure 1.

3.1 Core Ledger and Consensus

At the foundation of the system is the blockchain core, which manages the ordered sequence of blocks and maintains the global state, including account balances, nonces, and processed transaction IDs. It enforces strict chain validation rules, executing genesis verification and stateful transaction checks across the entire ledger. Pending transactions are staged in a thread-safe mempool utilizing mutex-protected data structures to handle concurrent operations safely. Block finality is achieved through a Proof-of-Work (PoW) consensus mechanism, where miners iteratively compute SHA-256 hashes to satisfy a deterministic difficulty prefix.

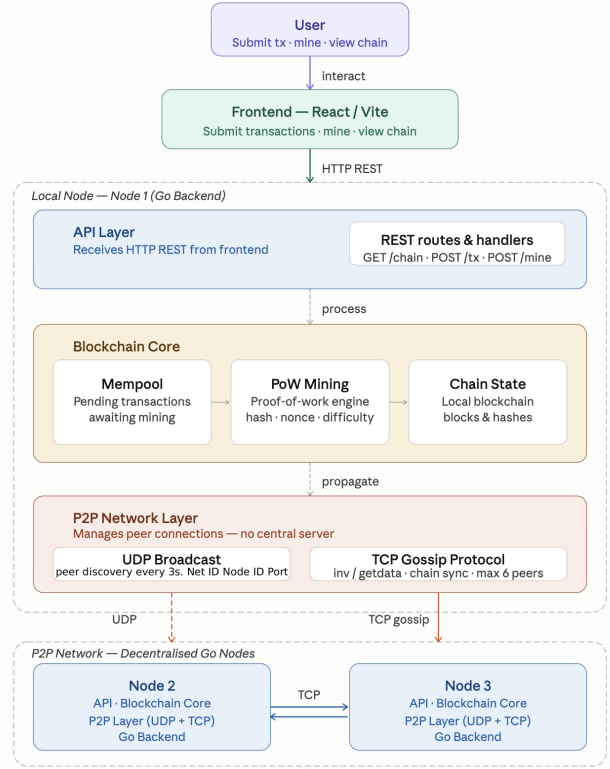


Figure 1: System architecture of the blockchain network, illustrating the data flow from the frontend through the local node's API, Blockchain Core, and P2P Network Layer to the decentralised peer nodes.

3.2 Cryptography and Transaction Model

The identity and authorization layer is governed by Elliptic Curve Cryptography (ECDSA) utilizing the P-256 curve. Wallet addresses are derived via SHA-256 hashing of the uncompressed public key, truncated to a 20-byte identifier. The transaction system handles the creation of genesis, coinbase, and standard transfer transactions. To ensure data integrity and non-repudiation, transfer transactions are secured with fixed-width 64-byte (r, s) signatures, validated against the canonical byte sequence of the transaction payload.

3.3 Peer-to-Peer Networking

Decentralized communication is handled by a custom P2P networking implementation. The architecture employs a dual-protocol approach: lightweight UDP broadcasts (emitted every three seconds) are used for local peer discovery, while reliable TCP connections manage continuous data exchange. Upon a successful TCP handshake, peers exchange known addresses to facilitate transitive discovery. The network state is synchronized using an inventory-based `inv/getdata` protocol for blocks and transactions, with strict limits on connection counts (e.g., a maximum of six peers) to prevent resource exhaustion.

3.4 Application Interface and Analysis

To bridge the internal blockchain logic with external users, the system exposes a comprehensive HTTP REST API. This interface provides endpoints for state queries, mempool access, transaction submission, and mining triggers. Interfacing with this API is *Block-View*, a React-based frontend explorer that enables real-time chain visualization and node management. Finally, the system design incorporates an isolated security testing module. This suite programmatically executes adversarial scenarios including replay attacks, signature forgery, and state overdrafts against a fresh blockchain instance to actively verify the network's cryptographic resilience.

3.5 Initial Token Distribution and Faucet Implementation

To facilitate immediate transaction validation and system testing within our development environment, the architecture implements the distribution mechanisms described in the background.

Upon initialization, the blockchain core generates a hardcoded Genesis block. This block contains a specialized coinbase transaction that executes the pre-mining phase by minting a predetermined supply of tokens. These tokens are allocated directly to a centralized treasury address, as well as to several pre-configured test accounts (e.g., "Alice" and "Bob"). This setup allows users to immediately interact with the system, execute peer-to-peer transfers, and observe validation logic upon startup without the need to initialize multiple nodes or perform initial mining.

To distribute tokens from the treasury to new external accounts, the system exposes a Faucet via the `POST /faucet` HTTP API endpoint. When an external account requests funds, the Faucet component constructs and signs a standard transfer transaction from the treasury to the user's derived wallet address. This transaction is then broadcast to the mempool, ensuring that the distribution of funds is subject to the exact same cryptographic validation and mining sequence as standard peer-to-peer transactions.

4 Implementation

This section details the practical realization of our decentralized blockchain system, outlining the structures, protocols, and architectural decisions made during development. The core blockchain node, state machine, cryptographic validation, and peer-to-peer networking modules are implemented entirely in Go, leveraging its native support for concurrency and robust standard networking library. The interactive user interface, real-time block explorer, and isolated security evaluation panel are realized as a modern single-page frontend application utilizing React.js, compiled and served via the Vite build tool. Together, these layers form a cohesive, runnable system that demonstrates the security and decentralized synchronization properties of a production-like blockchain.

4.1 Block Structure and Hashing

Each block is represented as a Go struct containing seven fields:

Index: Sequential position in the chain.

Timestamp: Unix time in milliseconds.

Transactions: A slice of Transaction structs.

PrevHash: The SHA-256 hash of the preceding block.

Hash: The block's own computed hash.

Nonce: The Proof-of-Work solution.

Difficulty: The required number of leading zero hex characters.

The hash is computed by serializing the relevant block fields into a deterministic pipe-delimited string:

```
record = fmt.Sprintf("%d|%d|%s|%s|%d|%d",
    block.Index, block.Timestamp,
    json.Marshal(block.Transactions),
    block.PrevHash, block.Nonce,
    block.Difficulty)
hash = SHA-256(record)
```

This approach ensures that any modification to any field, including the transaction contents, produces a completely different hash, providing immediate tamper evidence. By serializing the entire transaction array directly into the block record, our implementation deliberately simplifies the architecture. It bypasses the need for Merkle Trees, commonly used in production networks like Bitcoin in favor of a more straightforward validation model suitable for our scope.

4.2 Proof-of-Work Implementation

The Go-based blockchain integrates a Proof of Work (PoW) consensus mechanism where nodes computationally compete to find a SHA-256 block hash meeting a strict difficulty prefix. This mechanism fundamentally protects the network from data tampering and ensures stable node synchronization. To satisfy the consensus rules, a mutable nonce (N) is iteratively incremented with one and hashed alongside the block's core attributes: Index (I), Millisecond Timestamp (TS), JSON-serialized Transactions (TX), Previous Block Hash (P), and Difficulty (D), concatenated deterministically using a pipe delimiter (`|`). The mining loop continues until the resulting hexadecimal hash exhibits a prefix of at least D consecutive leading zeros, which mathematically constrains the hash value below a target threshold determined by the difficulty:

$$H(B) = \text{SHA-256}(I \mid TS \mid TX \mid P \mid N \mid D) < 16^{64-D}$$

4.3 Transaction Signing Workflow

When a user initiates a transaction via a wallet, the system executes the following steps in `NewSignedTransferTransaction`:

- (1) **Canonical Serialization:** The transaction attributes, consisting of the sender's address (`From`), recipient's address (`To`), value (`Amount`), millisecond timestamp (`Timestamp`), transaction class (`Type` as "transfer"), and current account sequence number (`Nonce`), are structured into a map. In `SigningBytes()`, this payload is serialized into JSON. Go's standard JSON library deterministically marshals the map keys in alphabetical order, producing a canonical byte array:

```
payload = {"amount":A, "from":F, "nonce":N,
    "timestamp":T, "to":R, "type":"transfer"}
```

- (2) **Hashing the Payload:** The canonical JSON byte sequence is hashed using the SHA-256 algorithm from Go's standard

crypto/sha256 package, to produce a 32-byte message digest.

- (3) **ECDSA Signature Generation:** The 32-byte digest is signed using the sender's private key via the standard crypto/ecdsa package. This produces two coordinate integers, (r, s) . In SignBytes, these coordinates are encoded as fixed-width 32-byte integers and appended together to form a raw 64-byte cryptographic signature:

$$\text{Sig} = \text{pad}_{32}(r) \parallel \text{pad}_{32}(s)$$

This 64-byte signature is hex-encoded into a 128-character string and stored in the transaction's Signature field.

- (4) **Public Key Attachment:** The sender's uncompressed ECDSA public key, represented in the standard uncompressed format (65 bytes: a 0x04 prefix byte followed by the 32-byte X and Y coordinates), is hex-encoded into a 130-character string and stored in the transaction's PublicKey field.
- (5) **Transaction ID Computation:** Once fully signed, the unique transaction identifier (ID) is computed in CalculateTransactionID by taking the SHA-256 hash of a pipe-delimited string of all fields, including the public key and signature:

```
record = "From|To|Amount|Timestamp|Type|Nonce|
PublicKey|Signature"
tx.ID = Hex(SHA-256(record))
```

4.4 Transaction Verification Workflow

When a node receives a transaction (either through the REST API or propagated via the P2P network), it runs stateful and stateless checks in ValidateTransaction before staging it in the mempool:

- (1) **Address Derivation Check:** The node decodes the transaction's hexadecimal PublicKey and verifies that it derives the sender's address. In AddressFromPublicKey, the 65-byte uncompressed public key is hashed using SHA-256, and the first 20 bytes of the digest are hex-encoded to match the From address:

```
DerivedAddress = Hex(SHA-256(PublicKey)0...19)
```

If the derived address does not match tx.From, the transaction is rejected.

- (2) **Signature Validation:** The node parses the 64-byte signature back into r and s components, reconstructs the identical canonical signing JSON bytes, hashes them with SHA-256, and validates the signature against the decoded P-256 public key using ecdsa.Verify. Any corrupted signature (tested by XOR-ing bits) causes immediate rejection.
- (3) **Stateful Sequence and Balance Verification:** The transaction's Nonce is compared against the expected next sequence number for the sender's address ($\text{NextNonce} = \text{LastNonce} + 1$). The transaction amount is also checked against the sender's accumulated ledger balance. If the nonce is incorrect or if the sender has insufficient funds (an overdraft attempt), the transaction is immediately discarded.

4.5 Chain Validation

To guarantee that all nodes arrive at an identical ledger state without a central coordinator, the ValidateChain function in blockchain.go rebuilds the entire ledger state in memory starting from the genesis block. This process tracks account balances (balances map[string]int), account nonces (nonces map[string]uint64), and unique transaction IDs (seenTxIDs map[string]bool) through the following sequential validation pipeline:

- (1) **Genesis Validation:** The process verifies that the first block (index 0) has a sequential index and a valid PoW hash prefix. It invokes ValidateGenesisTransactions to ensure all transactions in the genesis block are of type "genesis" with empty sender addresses and nonces of 0. Its allocations are then applied to initialize account balances.
- (2) **Header Sequencing and Hash Chaining:** For every subsequent block, the stateless IsBlockValid helper verifies that the block index is contiguous ($\text{curr.Index} = \text{prev.Index} + 1$), the PrevHash matches the preceding block's hash, and the current hash is correctly computed and meets the PoW difficulty target. Additionally, it ensures the timestamp is monotonically increasing and within a 2-hour future window limit.
- (3) **Stateful Transaction Validation:** Transactions inside each block are validated against the accumulated ledger state using ValidateBlockTransactionsAgainstState. This step enforces all stateless and stateful validations, including ECDSA signature verification, address derivation, nonce sequencing, and balance/overdraft checks as detailed in the *Transaction Verification Workflow* subsection. It also guarantees double-spend protection by rejecting duplicate transaction IDs across the chain. Additionally, coinbase transactions are verified to ensure exactly one reward transaction is present per block, positioned first, with an empty sender and nonce, and not exceeding the configured MiningReward.
- (4) **State Application:** If all validations succeed, the transacted amounts are subtracted from the senders, added to the recipients, the account nonces are updated, the transaction IDs are marked as seen, and the loop proceeds to the next block until the entire chain is verified as valid.

4.6 Peer-to-Peer Protocol

Decentralized communication is handled by a custom peer-to-peer networking implementation. The architecture employs a dual-protocol approach to separate node discovery from reliable data exchange, a three-stage progression visualized in Figure 2.

Decentralized communication is handled by a custom peer-to-peer networking implementation. The architecture employs a dual-protocol approach to separate node discovery from reliable data exchange. Local peer discovery is facilitated by lightweight UDP broadcasts emitted every three seconds. When a node detects a broadcast from an unknown peer operating on the same network identifier, it initiates a reliable TCP connection.

Upon connection, nodes execute a strict handshake protocol. During this initial exchange, they verify network compatibility,

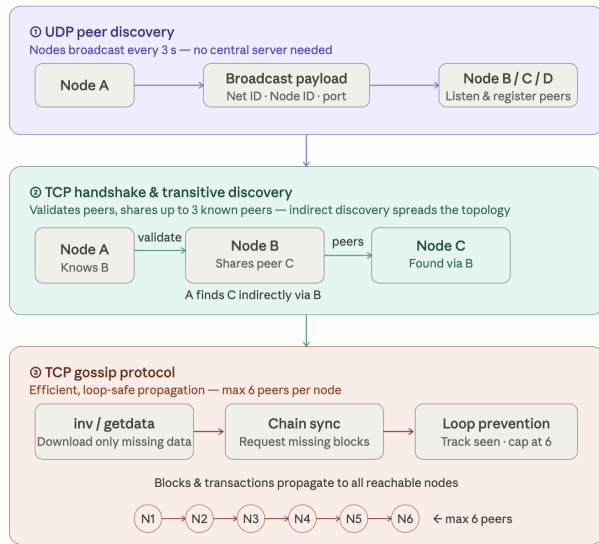


Figure 2: The three-stage peer-to-peer networking protocol, illustrating UDP peer discovery, TCP handshakes for transitive routing, and the inventory-based TCP gossip protocol.

such as matching Genesis hashes, and share state metadata, including their current chain height. To promote network robustness and transitive discovery, the handshake payload also includes up to three randomized, known peer addresses. This enables newly joined nodes to rapidly build a localized topology without relying on a central directory. To mitigate resource exhaustion and prevent network flooding, nodes strictly enforce a connection limit of six simultaneous peers.

Data propagation utilizes an efficient, inventory-based gossip protocol formatted as newline-delimited JSON. Rather than blindly broadcasting large payloads, nodes first announce the availability of new blocks or transactions via an *inv* (inventory) message containing only the respective cryptographic IDs. The receiving node cross-references these IDs against an internal, thread-safe *SeenTx*s and *SeenBlocks* cache. If the item is unknown, the node explicitly requests the full payload via a *getdata* message, subsequently receiving the actual tx or block object. Furthermore, for initial synchronization or resolving chain forks, a node discovering that its peer has a higher chain height during the handshake will issue a *getchain* request to download and evaluate the peer’s full ledger.

4.7 HTTP REST API

To bridge the internal peer-to-peer network with external clients, each blockchain node exposes a comprehensive HTTP REST API. This interface serves as the primary gateway for user interactions, effectively decoupling the underlying Go backend from the user interface. The API provides read-only endpoints for querying the global state, such as fetching the current chain, checking wallet balances, and inspecting the mempool. Additionally, it exposes state mutating endpoints that allow clients to submit signed transactions

(`POST /transactions`), request pre-mined tokens (`POST /faucet`), and trigger the Proof-of-Work mining process (`POST /mine`).

4.8 Frontend Explorer and Security Lab

Interfacing directly with the REST API is *BlockView*, a Single-Page Application built with React and Vite. The frontend is designed to connect to one or more node APIs simultaneously, providing users with a real-time, interactive dashboard of the decentralized network. The application architecture is designed around network monitoring, chain visualization, and active node control.

Through the interface, users can monitor the live status of connected nodes via automatic polling, tracking metrics such as current chain height, mempool size, and active peer counts. The ledger itself is rendered as an interactive visual chain, allowing users to inspect detailed block metadata and individual transaction payloads. To facilitate active network participation, the application provides dedicated modules for requesting faucet tokens, constructing and signing ECDSA transfer transactions, and initiating mining operations.

Furthermore, the frontend incorporates an isolated *Security Lab*. This specialized testing interface allows users to programmatically execute adversarial simulations including replay attacks, signature forgery, and state overdrafts against the connected node. The Security Lab visualizes the step-by-step execution flow of each attack, explicitly highlighting the cryptographic defense mechanism triggered and the actual error response returned by the backend system.

5 Evaluation

5.1 Experimental Setup

All experiments were conducted on a local machine running multiple blockchain node instances. Each node was started as a separate process with distinct API and P2P ports. The default configuration uses difficulty 3, a mining reward of 50 coins, and genesis allocations of 1000 (faucet), 100 (Alice), 50 (Bob), and 25 (Charlie) coins. The Go implementation was compiled with Go 1.22, and the frontend was served via Vite’s development server.

5.2 Experiment 1: Chain Integrity (Tampering Detection)

We modified a block’s transaction data after mining and re-ran chain validation. The system correctly detected the tampering because the stored hash no longer matched the recomputed hash over the modified data. The *ValidateChain* function returned: “invalid block hash”. Additionally, even if the attacker recalculates the tampered block’s hash, the next block’s *PrevHash* field would not match, causing cascading validation failures. This demonstrates the tamper-evident property of hash-linked chains.

5.3 Experiment 2: Transaction Validation (Attack Resistance)

We implemented a comprehensive attack suite (*attack_api.go*) that runs five distinct attack scenarios against an isolated blockchain instance. Each attack tests a specific security mechanism:

Attack	Defense Mechanism	Result
Replay	Duplicate TX ID check	Blocked
Invalid Signature	ECDSA P-256 verification	Blocked
Wrong Public Key	Address derivation check	Blocked
Overdraft	State-based balance check	Blocked
Future Timestamp	2-hour timestamp limit	Blocked

Table 1: Attack resistance test results. All five attacks were correctly rejected by the system.

Replay Attack. The same signed transaction was submitted twice. The first submission succeeded, but the second was rejected with "transaction already exists in mempool", demonstrating that the duplicate transaction ID check prevents replay.

Invalid Signature. A valid transaction's ECDSA signature was corrupted by XOR-ing all bytes with `0xFF`. The system rejected it with "invalid signature", confirming that the P-256 ECDSA verification catches tampered signatures.

Wrong Public Key. A transaction was signed with Alice's private key but claimed Eve's public key and address. The system rejected it because the address derived from Eve's public key did not match the signature produced by Alice's key: "invalid signature".

Overdraft. A transaction attempting to send balance + 9999 coins was rejected with "insufficient balance", confirming that the stateful balance check prevents creation of value out of nothing.

Future Timestamp. A block with a timestamp 3 hours in the future was mined with valid PoW but rejected with "block timestamp is too far in the future", demonstrating the 2-hour timestamp validation window.

5.4 Experiment 3: Multi-Node Synchronization

We started three nodes on separate ports and verified synchronization behavior:

- (1) Nodes discovered each other via UDP broadcast within 3 seconds.
- (2) A transaction submitted to Node 1 appeared in Node 2 and Node 3's mempools via the `inv/getdata` protocol.
- (3) A block mined on Node 1 was propagated to and accepted by all other nodes.
- (4) A new node joining after block creation correctly synced the full chain via the `getchain/chain` protocol.
- (5) Peer discovery was transitive: Node 3, which only directly discovered Node 2, learned about Node 1 through the handshake's `known_peers` field.

6 Discussion

The system successfully demonstrates the core principles of blockchain technology through a working, end-to-end implementation. Several observations emerge from our evaluation:

Security coverage. The attack resistance tests show that even a simplified blockchain can achieve meaningful security guarantees through proper cryptographic design. The combination of ECDSA signatures, address derivation, nonce ordering, balance checks, and

timestamp validation creates a layered defense that catches qualitatively different attack vectors.

Networking limitations. The UDP broadcast discovery works well on a local network but would not function across network boundaries without modifications (e.g., using multicast or a bootstrap node registry). The ephemeral TCP connections used for broadcasting create overhead compared to persistent connections. The system also lacks mechanisms for handling network partitions gracefully, which could lead to temporary chain forks that would need to be resolved via the longest-chain rule.

Comparison to production systems. Our implementation omits several features present in production blockchains: persistent storage (our chain exists only in memory), Merkle trees for efficient transaction verification, UTXO or account-based state models, dynamic difficulty adjustment, block size limits, transaction fees, and smart contracts. However, the core architectural patterns — hash-linked blocks, PoW mining, signed transactions, mempool staging, and gossip-based propagation — are faithfully represented.

7 Related Work

Bitcoin [5] introduced the first decentralized cryptocurrency, combining hash-linked blocks with Proof-of-Work consensus to achieve trustless agreement. Ethereum [?] extended this model with a Turing-complete virtual machine enabling smart contracts. Our implementation draws inspiration from Bitcoin's UTXO-like balance computation and its gossip-based networking protocol (`inv/getdata`), while simplifying the consensus to basic longest-chain selection without difficulty adjustment.

Several educational blockchain implementations exist. Narayanan et al. [6] provide a textbook treatment of Bitcoin's architecture but without a runnable system. Ivan Kuznetsov's "Building Blockchain in Go" tutorial [?] covers persistence and basic networking but omits ECDSA signing and attack testing. Our work differentiates itself by providing a complete, full-stack system with a modern frontend, automated security testing, and a P2P layer with UDP discovery, handshake-based peer exchange, and inventory-based propagation.

8 Conclusion

We have designed, implemented, and evaluated a modular blockchain system in Go with a React-based frontend explorer. The system faithfully implements the core mechanisms of decentralized ledger technology: SHA-256-based block linking, configurable Proof-of-Work consensus, ECDSA-signed transactions with nonce-based replay protection, a transaction mempool, and a peer-to-peer networking layer with UDP discovery and TCP communication.

Our evaluation demonstrates that: (1) mining time scales exponentially with PoW difficulty as predicted by theory; (2) hash-linked blocks provide effective tamper detection; (3) the combination of cryptographic signatures, address derivation, nonce ordering, balance validation, and timestamp checks successfully defends against five distinct attack vectors; and (4) multiple nodes can discover each other, synchronize chain state, and propagate transactions and blocks without centralized coordination.

The key insight from this work is that a relatively compact codebase (approximately 2,500 lines of Go plus 1,500 lines of React) can

reproduce the essential security and consensus properties of production blockchain systems, while remaining transparent enough for educational analysis and experimentation.

9 Future Work

Several extensions would enhance the system's realism and capabilities:

- **Persistent storage:** Adding disk-backed block storage using a key-value store (e.g., BoltDB) to enable node restarts without chain loss.
- **Dynamic difficulty adjustment:** Implementing automatic difficulty scaling based on recent block times, as in Bitcoin's 2,016-block adjustment window.
- **Merkle trees:** Adding Merkle root computation to block headers for efficient transaction inclusion proofs without downloading the full block.
- **Persistent P2P connections:** Replacing ephemeral TCP connections with long-lived connections to reduce handshake overhead and enable bidirectional communication.

- **Fork resolution:** Implementing explicit orphan block handling and fork detection to improve behavior under network partitions.
- **Transaction fees:** Adding fee-based transaction prioritization to the mempool to simulate economic incentives for miners.

References

- [1] Binance Academy. n.d. Mempool - Glossary. <https://www.binance.com/en/academy/glossary/mempool> Accessed: 2026-05-18.
- [2] Binance Academy. n.d. What Is a Crypto Faucet? <https://www.binance.com/en/academy/articles/what-is-a-crypto-faucet> Accessed: 2026-05-18.
- [3] Binance Academy. n.d. What Is Pre-mining? <https://www.binance.com/en/academy/glossary/premining> Accessed: 2026-05-18.
- [4] Cynthia Dwork and Moni Naor. 1992. Pricing via processing or combatting junk mail. In *Advances in Cryptology—CRYPTO'92*. Springer, 139–147.
- [5] Satoshi Nakamoto. 2008. Bitcoin: A peer-to-peer electronic cash system. (2008).
- [6] Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller, and Steven Goldfeder. 2016. *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. Princeton University Press.
- [7] National Institute of Standards and Technology. 2015. Secure Hash Standard (SHS). FIPS PUB 180-4. doi:10.6028/NIST.FIPS.180-4